

OS-PRACTICAL-2

Develop a C program that uses the system calls `open()`, `read()`, `write()`, and `close()` to copy the contents of one file to another. Explain how control transitions between user space and kernel space during execution. Use the `strace` utility to trace all system calls generated by the command `cat sample.txt`. Analyze the sequence of system calls and identify the kernel services involved.

```
rishita@Rishitha:~$ touch os.txt
rishita@Rishitha:~$ cat >os.txt
Hey
^C
rishita@Rishitha:~$ nano os.c
rishita@Rishitha:~$ nano os.c
rishita@Rishitha:~$ gcc os.c
rishita@Rishitha:~$ gcc os.c -o os
rishita@Rishitha:~$ ./os
file copied successfullyrishita@Rishitha:~$ cat os.txt
Hey
rishita@Rishitha:~$ |
```

The program uses `open()` to open the source and destination files. `read()` reads the contents of the source file into a buffer, and `write()` writes the contents from the buffer into the destination file. Finally, `close()` closes both files. The output confirms that the file was copied successfully, and the contents of `copy.txt` were

verified using cat.

[illegible]

Used the strace utility with the command `cat sample.txt` to observe the system calls generated during execution. The strace output shows that the `cat` command accesses the file using system calls such as `openat()` and then transfers the file contents to the terminal. The file is finally closed using `close()`. This shows that when a user-space command needs to access a file, it requests the required services from the Linux kernel through system calls.

open() → opens sample.txt

read() → reads data

write() → displays data

close() → closes the file

